

Voice-Controlled Object Pick and Place for Collaborative Robots Employing the ROS2 Framework

Asha C S

*Department of Mechatronics
Manipal Institute of Technology,*

Manipal Academy of Higher Education, Manipal, 576104
0000-0002-9039-1548

Jeane Marina D'Souza

*Department of Mechatronics
Manipal Institute of Technology,*

Manipal Academy of Higher Education, Manipal, 576104
0000-0002-4861-3311
**Corresponding author*

Abstract—Industrial automation has heavily relied on robot arms, mainly when human involvement is not feasible. This paper introduces a robotic arm designed to pick up objects based on a patient's verbal command, precisely the word "pick." The system incorporates a perception unit for goal identification, a motion planner, and a robot controller. ROS2 is the middleware for context processing, a speech-based user interface using VOSK, and MoveIt2 for motion planning and control. The VOSK model has been implemented for speech recognition, demonstrating encouraging results through transfer learning from pre-existing weights. A finite state machine (FSM) has been developed to manage the execution flow of the route to the collaborative robotic arm, enabling it to carry out tasks effectively.

Index Terms—Collaborative Robots, Voice Control, Object Detection, Speech Recognition, Finite State Machine

I. INTRODUCTION

Collaborative robots, commonly referred to as Cobots, have recently gained popularity in various industries, such as installation, assembly, and disassembly. These robots offer enhanced safety features, making them ideal for collaborative tasks between humans and robots, particularly for individuals with physical disabilities. By integrating natural language into existing robotic systems, the intelligence of these machines is significantly improved, facilitating effective communication between humans and machines for collaborative work.

Voice-based interaction, combined with computer vision, can significantly enhance human-robot interaction. Cobots are recognized for their flexibility, easy installation, and quick setup, making them suitable for use on production lines and in the service industry. The safety features of Cobots are a significant improvement over traditional industrial arms.

Human interaction with Cobots can be enhanced through voice or visual control. With voice control, users can instruct Cobots to perform tasks on objects using voice commands. The Cobot then interprets the voice input through a vision-based approach and takes appropriate action. Voice control can be categorized into intent matching and keyword matching. Intent matching, similar to conversational AI, listens to commands and attempts to decipher the user's intent. In contrast, keyword-matching voice control searches for specific

keywords, such as "pick" or "place," to perform a particular task.

II. LITERATURE REVIEW

In recent years, software for robotic arms has made significant advancements, particularly with the introduction of the Robot Operating System 2 (ROS2) [1] and Moveit2 [2]. These developments have raised software standards, enabling robots to execute complex tasks with relatively less code. However, despite its evolution, the Moveit 2 and ROS 2 ecosystem still lacks features such as higher-level semantic perception and user-friendly interfaces.

User interfaces have also progressed, with OpenAI releasing its Whisper [3] model and the VOSK API [4]. These advancements provide speech-to-text models that facilitate a speech-recognition user interface. ROS 2 [5], which uses Data Distribution Services (DDS) [6] to manage communication. This departs from its predecessor, ROS, which uses a plain TCP protocol. DDS supports UDP and TCP protocols and allows for several custom Quality of Service (QoS) settings.

The authors in [7] use voice control to control the armchair robot with 80% accuracy. The paper [8] discusses voice control of a mobile robot that uses a deep learning model for speech recognition. While authors used MFCC combined with ANN for voice recognition to control the car's headlights [9]. Authors in [10] use the voice, gesture and brain signals to control the humanoid robot. A Google Assistant is used to interface the voice signal with the robot. The initial phase involves the user interface, which is speech-based. It necessitates research to understand current models and determine their strengths and weaknesses. A few grading parameters are considered for any speech-to-text model, including accuracy, conversion speed, and pronunciation robustness.

Each potential model is tested against these parameters, and a model is selected based on the results. Observing the model selection, an interface must be constructed from the model to the middleware. This interface must meet two conditions: (i) agnostic to the type of user interface and (ii) capable of streaming interface commands in real-time.

A. Methodology

The process starts with a simulation to assess the system's performance, testing algorithms and the pipeline before deployment. It is essential to fine-tune the algorithms for physical accuracy before deploying them on a robot. Integrating the simulator with the middleware can streamline the testing and deployment of algorithms. The selection of a suitable simulator should consider its compatibility with the middleware. Accurate robot modelling, including its dimensions and end-effector, is crucial for the simulator to incorporate potential hardware parameters like motors and sensors.

The inverse kinematics algorithm is selected once the simulation is configured with a collaborative robotic arm. Critical parameters for choosing an inverse kinematics algorithm include the solving rate and time to generate a solution. The solve rate evaluates the algorithm's ability to calculate joint angles based on the end-effector's position within the robot arm's workspace. The inverse kinematics algorithm's interface takes a 3D pose of the end-effector as input and produces an array of floating-point integers as output, corresponding to the manipulator's joints. This information guides the selection of a motion planning algorithm, which can be categorized into joint and Cartesian space planning.

After determining the type of motion planning algorithm, a specific algorithm is chosen and tested with various inputs to evaluate its performance. Parameters considered during algorithm selection include time taken, algorithmic complexity, computation intensity, solve rate, precision of output plan, and memory consumption. The output of the motion planning algorithm feeds into the robot control algorithm to move the robot joints. The robot control utilizes multiple PIDs to operate the robot joints' motors, requiring tuning for both hardware and simulation. Finite State Machines (FSMs) and Behaviour Trees are employed in this paper to oversee the task planner's operation and failures at each stage. The architecture chosen, incorporating parameters, simplifies task design. The task planners are designed to be user-friendly and intuitive, reducing the need for extensive foundational engineering and system integration, thereby streamlining the construction process for improved efficiency.

Middleware encompasses software libraries and frameworks facilitating communication among diverse software components while maintaining consistent data structures. A comparative analysis of various middleware options was conducted, evaluating each based on specific parameters using a scale from 0 to 1. This evaluation aided in identifying the most suitable middleware solution for the research needs.

TABLE I
MIDDLEWARE COMPARISON

Middleware	Resources	Simplicity	Supporting Libraries	Latency
CycloneDDS	0.5	0.5	0.1	1.0
ROS2 Humble	1.0	1.0	0.9	0.8
ROS1 Noetic	1.0	1.0	1.0	0.5

After consideration, ROS2 Humble with a Cyclone DDS backend was chosen as middleware.

In this experiment, a box is used for picking and placing purposes. Upon speech detection, a camera identifies its location, which directs the arm to move to the desired location.

B. Speech Recognition

Speech recognition encompasses two primary methods: intent recognition and literal speech-to-text. Intent recognition is employed when a command is directly derived from speech based on the intended meaning, training the model to understand the user's intent rather than their exact words. On the other hand, speech-to-text translates the user's spoken words into text, which an external interpreter interprets to align with the intended meaning.

Speech-to-text is preferred in this study due to its reliability, predictability, and repeatability. Since speech directly controls the robot, predicting the intent can be risky. Therefore, a more straightforward but safer algorithm is utilized to identify the text and match the string to check for specific keywords that form part of the intent. For example, if the user says "pick", it is automatically assumed to be a picking task. However, a limitation of this approach is that even if the user says, "Do not pick", the robot will still execute the picking task. Therefore, users are advised to use simple, directly interpretable commands without negatives, ensuring the accuracy and reliability of the robot's response to speech inputs.

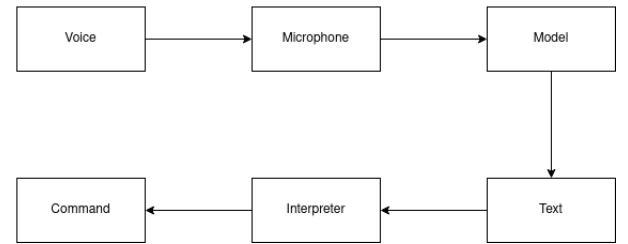


Fig. 1. Speech Recognition Flow

The dataset influences the complexity of the model, and even though the language is set to English, the model's task is not simplified due to the various pronunciation methods, which can confuse the model if it's not trained on a specific accent. Regarding speech recognition, background noise and real-time capabilities are critical factors. Background noise is addressed by a filtering process, and once the speech input is given, if the voice intensity exceeds a threshold, it's considered user speech and converted to text. The system is multithreaded, enabling concurrent audio listening and speech-to-text, allowing the system to listen to critical commands such as "place" in real-time.

Speech recognition involves three steps: capturing audio from the microphone, transcribing the captured audio, and concurrently listening to words. Audio transcription is achieved by

recording audio segments, temporarily storing them in a cache, transcribing them, and then clearing the cache. The storage and transcription process depends on the size of the chunk, and command listening must be variable, so the chunk is recorded as long as the audio remains above a certain threshold. However, only one process can access the microphone at a time. To avoid implementing complex workflows, such as a scheduler, to allocate resources to both methods, 1-second chunks are taken from the microphone when not listening to commands. Transcription via VOSK [4] is a direct process where a deep learning model uses the .wav file. The primary challenge lies in the accent, as speakers can have different accents for the same language, necessitating the model's training with the intended accent.

C. Task Planning

In task planning, the primary consideration is the simplicity of the task design. For this reason, we have chosen Finite State Machines over Behaviour Trees. While Behaviour Trees are more suitable for intricate task designs, Finite State Machines are selected for their brevity. A notable advantage is that Behaviour Trees can be substituted with Finite State Machines in the pipeline, although this is beyond the purview of an essential pick-and-place task.

State management is to make the system exist and perform certain functions in a particular state. The state machine is to manage the properties and I/O of the system in a modular way, as shown in Figure 2.

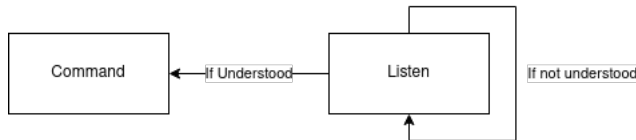


Fig. 2. State Management Example

Finite State Machines are frameworks that aid in knowing the flow of construct execution. They help build tasks between available software interfaces in our scenario. We will make a robot arm pick up a box for this case. We inherit all states with state base class, exploiting polymorphism; whenever a state change has to be made, a shared pointer previously defined in the machine/executor will be made with the appropriate child class. Each state is a ROS Node that does all its functions asynchronously and needs to have the **OnEnter** and **OnExit** functions activated by the executor. There are four parameters for selecting a simulator for the work. Based on the research and testing done, here are the results of each simulator graded under each parameter ranging from 0 to 1:

Gazebo Simulator sets the best score in the Goldilocks's zone. It is not resource intensive and provides an average photo-realism and good physics accuracy. It also has excellent middleware support with ROS 2 Humble. Once the plugins are loaded, we use the xacro system to convert our robot from xacro format to URDF format and then to SDF format. This

can then be given to a program that would access the ECS system and load an entity concerning the SDF generated in the Gazebo World.

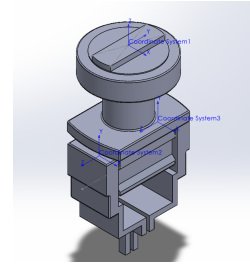


Fig. 3. UR5e gripper to pick and place the object

Upon doing this, we receive the following output:

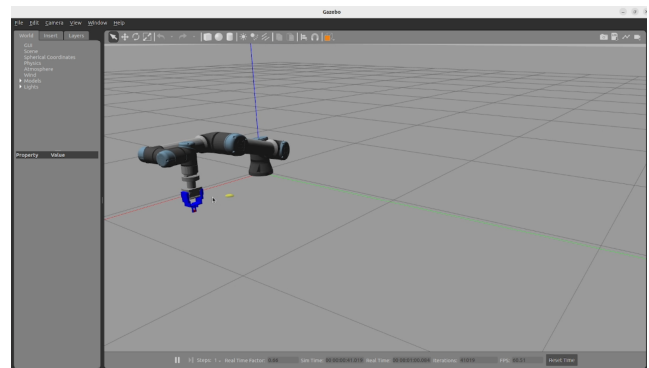


Fig. 4. UR5e picking an object

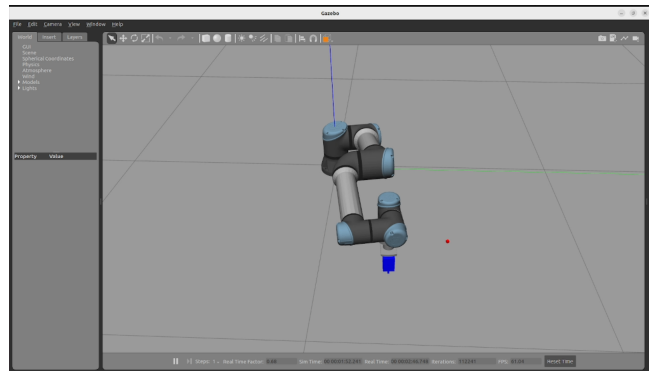


Fig. 5. UR5e placing an object

D. Robot Kinematics

Robot kinematics includes forward and inverse kinematics. Forward kinematics is used to get the end-effector position with joint states as input, while inverse kinematics calculates the possible joint angles based on the end-effector pose.

The approach of Denavit-Hartenberg (DH) Parameters is utilized for forward kinematics. DH parameters compute the homogeneous transformation matrix from the base link to the end-effector. This convention is employed to assign frames to benefit serial-manipulator chains. The DH parameters for a

single joint include the joint angle, link twist, joint offset, and link length. The link length and twist are permanently set, but the type of joint determines whether the joint angle and offset are fixed or variable. In the case of a prismatic joint, the joint offset is variable, and the joint angle is fixed. Conversely, the joint angle is variable for a revolute joint, and the joint offset is fixed.

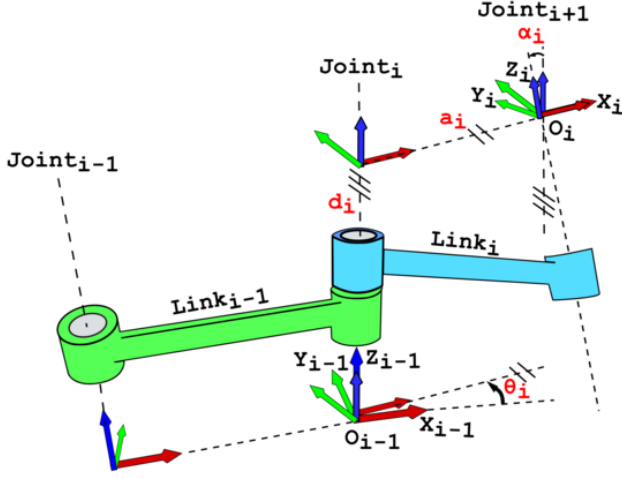


Fig. 6. DH Parameters between two links

In Figure 6:

- d is a joint offset. It is the offset between joint i and joint $i - 1$.
- a is the link length. It is the length of the link i .
- α is the link twist. It is the twist of link i .
- θ is the joint angle. It is the angle of joint i .

This would then form the transformation matrix per joint-link pair as:

$$T_n^{n-1} = \begin{bmatrix} \cos \theta_n & -\sin \theta_n \cos \alpha_n & \sin \theta_n \sin \alpha_n & a_n \cos \theta_n \\ \sin \theta_n & \cos \theta_n \cos \alpha_n & -\cos \theta_n \sin \alpha_n & a_n \sin \theta_n \\ 0 & \sin \alpha_n & \cos \alpha_n & d_n \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1)$$

We form matrices for each joint-link pair and multiply them all to get the final matrix for the transformation between the base link and end-effector. The DH parameters for a UR5e are:

TABLE II
UR5E DH PARAMETERS

Joints	Joint Angle (θ)	Joint Offset a	Link Length d	Link Twist α
Joint 1	var	0	0.151	1.57
Joint 2	var	-0.24	0	0
Joint 3	var	-0.21	0	0
Joint 4	var	0	0.131	1.57
Joint 5	var	0	0.08	-1.57
Joint 6	var	0	0.09	0

All the joint angles are variable because the UR5e is a 6 DOF robot arm with all revolute joints. The kinematic diagram of the is:

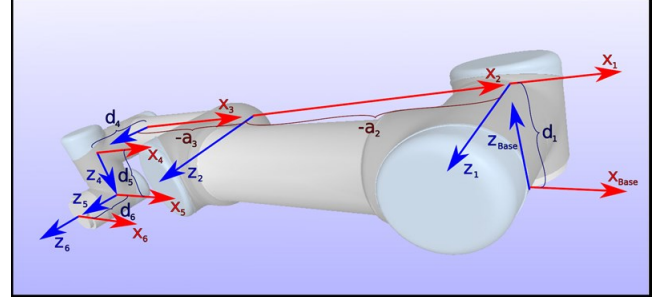


Fig. 7. Kinematic diagram of the UR5e.

For Inverse kinematics, we decided based on the solve rate and the average time to generate a solution. The plugin Track-IK [11] has proved itself to be the best amongst its competitive general inverse kinematics algorithms over the years. According to their data, here is a comparison of Track-IK with two other kinematics algorithm competitors:

TABLE III
IK ALGORITHMS COMPARISON

Algorithm	Solve Rate (%)	Average Time Taken (ms)
KDL	16.52	4.21
KDL-RR	88.58	0.74
Trac-IK	99.15	0.37

The Trac-IK algorithm leverages Sequential Quadratic Programming (SQP) based nonlinear optimization. It converges to an optimal solution or solutions using a threaded program model and gives fast results. It is adaptable to any motion planning approach because of its low average time taken, and it integrates easily with ROS and MoveIt because of the efforts at Trac-labs.

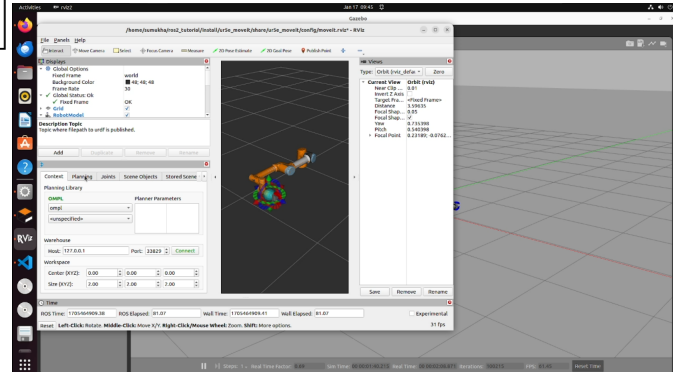


Fig. 8. Inverse Kinematics Visualization in RViz.

E. Motion Planning

Motion planning is a methodology to generate plans in a way that the lower-level controller has to follow its requests.

It considers workspace and joint constraints and generates a plan that moves the robot from configuration A to B. There are two primary motion planning approaches: joint and task space planning.

Joint Space planning involves the motion planning algorithm creating a space with dimensions equivalent to the robot's joints and then attempting to converge on a valid motion plan within this constructed space. The benefit of this approach is that it limits the runs of the inverse kinematics algorithm, thereby reducing usage and potential points of failure. However, the downside is that the construction of the space becomes more complex with the addition of joints. The space is already too intricate for a 6 DOF robot manipulator to construct and interpret the environment.

This method is overly computationally demanding and complex for an intelligent robot, but it could be more effective with an alternative strategy and in controlled environments, such as automation.

Task space planning involves the motion planning algorithm creating a 3D space of the environment and searching within this space for a feasible plan. Once the plan is established, the Inverse Kinematics (IK) algorithm calculates the joint states at each stage and feeds them to the robot control. This approach has a shorter feedback loop, making the robot more responsive and eliminating the need for space construction computation. However, inverse kinematics is a drawback of this method. Even though the end-effector is well within the workspace, and it is geometrically possible to calculate a configuration for the robot, it may not always converge. This can lead to planning failures, resulting in probabilistic operation. Even if the IK algorithm has previously solved for an end-effector pose, it may fail due to minor changes in precision or alterations in existing precise values, as it is an optimization algorithm. However, these issues can be effectively managed with a robust software system and retry mechanisms. Given the relatively short time required to solve the IK, this is the most optimal solution selected for our work.

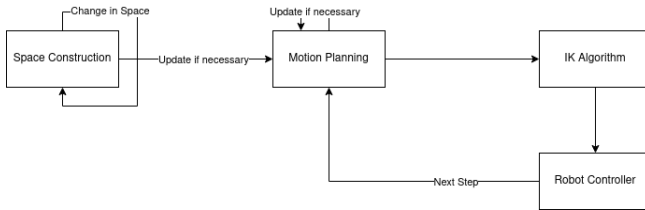


Fig. 9. Task Space Planning Working

1) Planning Algorithms: The planning algorithm accepts the current position, goal position, and environment as inputs and formulates a plan from the current to the goal position. We utilized an open-source library known as The Open Motion Planning Library [12], which offers sampling-based algorithms such as RRT, PRM, and others. We employed a meta-optimization planning strategy provided by OMPL, known as Anytime Path Shortening. This strategy operates multiple

planning libraries in OMPL's suite to achieve a suboptimal solution, demonstrating its effectiveness.

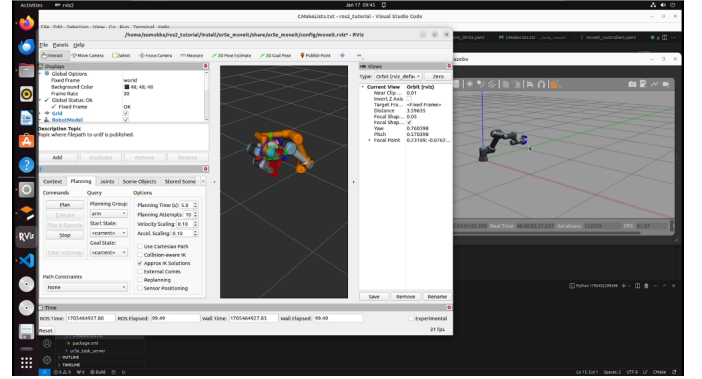


Fig. 10. OMPL Planning to reach configuration given environment constraints.

F. Robot Control

An array of PID controllers handles robot control. Each joint has its motor controlled by a PID controller that takes a joint angle as input and moves the motor and, consequentially, the joint into a desired position.

A PID controller is a proportional, integral, and derivative controller; a single joint using a PID can be represented by this formula:

$$e_i = \theta_f - \theta_{i-1} \quad (2)$$

$$\theta_i = k_p * e_i + k_i * \int_0^{i-1} e_i * dt + k_d * \frac{de_i}{dx} \quad (3)$$

where i is the next iterative step, and θ_f is the goal theta.

G. PID Controller

Adjusting the PID gains is crucial for the robot joints' responsiveness. A practical and safe PID tuning strategy involves setting the integral and derivative gains to zero. Maintaining a low proportional gain prevents overreaction and sets minor joint goals to observe non-adverse reactions. This approach enables the robot to move slowly between short and precise distances. Gradually increasing the proportional gain should facilitate a safe understanding of the appropriate proportional gain.

Derivative and Integral terms complement proportional control. Increasing the derivative gain will dampen the oscillations if the controller oscillates about the suitable θ_f . A derivative gain that is too high can slow the response time. Damping the oscillations can cause the controller to reach a steady state error. This occurs when the derivative gain is damping the proportional control equally. Increasing the integral term, accumulating error over time, will ensure that the controller reaches the goal even in steady-state conditions. Integral gain is always low. It can cause adverse reactions over time since the error is constantly accumulating.

An anti-windup solution is employed to prevent the adverse reaction caused by the integrator. The simplest anti-windup

solution involves clamping the overall integral term's value, which prevents it from exceeding specific values and causing failures.

Emergency Stops are buttons accessible to anyone in the robot teach pendent. When pressed, it immediately stops all the joints in their position and remains as they are until rebooted safely. This is extremely helpful in stopping the robot from a collision if its observed trajectory is on course for a crash.

Trajectory scaling is the constraint of velocity, acceleration, and jerk. These constraints are max and min clamps that help keep the robot moving safely without random increases or decreases in speed or acceleration. These values are to be tuned and calibrated every so often to ensure unpredictable behaviour does not cause issues with the hardware.

III. RESULTS AND DISCUSSION

It was then observed that the model could detect a red box used for experimental purposes. However, if trained using a deep learning model, the user can select any objects for the pick and place operation.

The speech detection part primarily consists of a speech recognition API called VOSK [4], both interfaced with ROS 2 Middleware. This speech recognition module is configured with an asynchronous callback-based audio segment receiver featuring a sampling rate of 16000 Hz, an audio chunk size of 8000, float/int 16 precision, and a single channel.

Two options were considered: Whisper [3] and VOSK. Whisper, developed by OpenAI, was slower in transcription than VOSK. Additionally, Whisper was less accurate in transcriptions with higher background noise than VOSK. There was also a difference in accent recognition between the two; while Whisper could transcribe all accents, VOSK could not. For testing purposes among the developers, VOSK was set to English.

Whisper was not chosen for our work due to its inability to provide accurate or quick transcriptions. It's worth noting that Whisper's original intent is for offline transcription across multiple languages and various accents.

Simulating the robot manipulator went successfully. Gazebo Fortress was used as a simulator, and the UR5e was spawned in and is fully controllable atop a table. The real-time factor of the simulator oscillates between 95% and 98%. This factor signifies the relationship between the simulators and the CPU clock. If both CPU and simulator clock run the same epoch, the real-time factor would be 100%

Robot planning is done with the help of OMPL. It uses the algorithm that uses different planners in multiple threads and tries to converge into a single output. The planner converges on a plan 7 out of 10 times in approximately 0.1s. This was calculated empirically by keeping the environment, the start, and the goal point constant.

IV. QUALITATIVE RESULTS

The work aimed to develop a universal robot arm capable of picking up and placing objects controlled through a user-friendly interface. The chosen interface was speech-based;

when the program receives spoken instructions, it triggers a finite state machine that identifies objects in the robot's vicinity, such as a box, and communicates this information to the finite state machine. This detection data is then input into the Inverse Kinematics algorithm to determine the joint angles required to complete the task. A robot motion planner then uses these joint angles to devise a motion plan, which the robot control reads and executes accordingly.

On the speech recognition front, the speech fails if the amplitude is too low; even with increased gain, VOSK does not transcribe the output correctly.

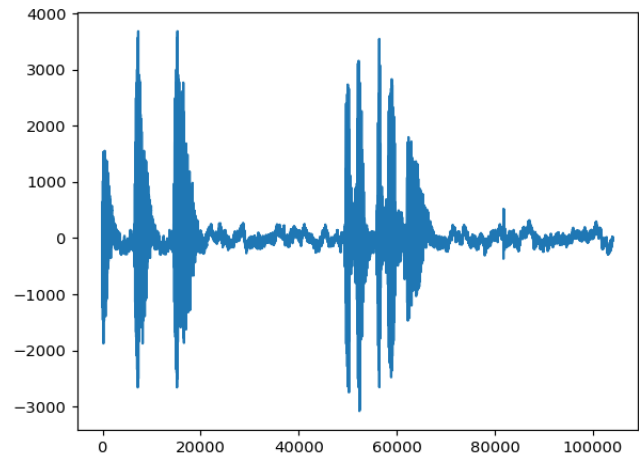


Fig. 11. A sample voice signal

VOSK encounters difficulties if the user speaks too rapidly or does not articulate the words clearly. This poses a challenge, as users cannot always be expected to pronounce their phrases or speak very slowly. The speech recognition library [13] is not compatible with multichannel or multi-array microphones, such as the built-in microphone in the Kinect camera. This presents a problem, as the camera's microphone was initially a convenient feature, and it would be beneficial to avoid having wired microphones in the setup. However, a powerful single-channel microphone is required since multichannel microphones are not functional.

Regarding motion planning, even when the same starting point, environment, and goal point are provided, the motion planning algorithm remains probabilistic. It functions correctly most of the time, occasionally requiring multiple retries to formulate a plan. While this is not ideal, it is an inherent characteristic of sampling-based algorithms, which are quick but probabilistic.

V. CONCLUSION

Moveit2 [2] assists in configuring the robot arm according to industrial standards. However, some software struggles to keep up with their documentation when using advanced middleware like ROS2 Humble [5]. We developed a universal pipeline for

collaborative robotic arms to exhibit the intelligence necessary for object detection. The robot required an accessible input form, which was provided through speech. The work involved using intent detection and searching for specific words in the speech, such as “place” or “pick”. The robotics aspect of the work entailed configuring a robot arm to use sampling-based algorithms for motion planning and a quadratic programming-based method for solving inverse kinematics. Task space planning was employed to ensure the robot’s reactivity. PID control was used to tune each joint for individual safety. Finite State Machines were utilized for proper task planning, allowing for the existence and transition between different states, such as pick, place, etc. Real-time object detection can identify specific objects, which will be addressed in future.

REFERENCES

- [1] M. Quigley, B. Gerkey, K. Conley, J. Faust, T. Foote, J. Leibs, E. Berger, R. Wheeler, and A. Ng, “Ros: an open-source robot operating system,” in *Proc. of the IEEE Intl. Conf. on Robotics and Automation (ICRA) Workshop on Open Source Robotics*, Kobe, Japan, may 2009.
- [2] P. Robotics. (2014) Moveit motion planning framework. [Online]. Available: <https://moveit.ros.org/>
- [3] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” 2021.
- [4] alphacep. (2022) Vosk. [Online]. Available: <https://github.com/alphacep/vosk-api>
- [5] D. Thomas, W. Woodall, and E. Fernandez, “Next-generation ROS: Building on DDS,” in *ROSCon Chicago 2014*. Mountain View, CA: Open Robotics, sep 2014. [Online]. Available: <https://vimeo.com/106992622>
- [6] I. Object Management Group. (1997) Dds data distribution services. [Online]. Available: <https://www.dds-foundation.org/>
- [7] M. Piyaneeanart and M. Ketcham, “Automatically moving robot intended for the elderly with voice control,” *International Journal of Online and Biomedical Engineering (iJOE)*, vol. 17, no. 06, pp. 19–48, 2021.
- [8] S.-A. Li, Y.-Y. Liu, Y.-C. Chen, H.-M. Feng, P.-K. Shen, and Y.-C. Wu, “Voice interaction recognition design in real-life scenario mobile robot applications,” *Applied Sciences*, vol. 13, no. 5, p. 3359, 2023.
- [9] W. Astuti, S. Tan, M. Solihin, R. Vincent, and B. Michael, “Automatic voice-based recognition for automotive headlights beam control,” *International Journal of Automotive and Mechanical Engineering*, vol. 18, no. 1, pp. 8454–8463, 2021.
- [10] Q. K. Pham, T. V. Vo, and P. T. Tran, “On the implementation of a low-cost mind-voice-and-gesture-controlled humanoid robotic arm using leap motion and neurosky sensor,” *Journal of Electrical Engineering & Technology*, vol. 17, no. 1, pp. 665–683, 2022.
- [11] P. Beeson and B. Ames, “Trac-ik: An open-source library for improved solving of generic inverse kinematics,” in *2015 IEEE-RAS 15th International Conference on Humanoid Robots (Humanoids)*, 2015, pp. 928–935.
- [12] I. A. Şucan, M. Moll, and L. E. Kavraki, “The Open Motion Planning Library,” *IEEE Robotics & Automation Magazine*, vol. 19, no. 4, pp. 72–82, December 2012, <https://ompl.kavrakilab.org>.
- [13] A. Zhang. (2023) Speech recognition. [Online]. Available: <https://pypi.org/project/SpeechRecognition/>